AI Code Translator and Explainer
Using Next.js and Ollama

A Research Paper Submitted in Partial Fulfillment of Academic Requirements

International Journal of Artificial Intelligence and Software Engineering (IJAISE)

Volume 12, Issue 3, 2025 | ISSN: 2456-7891

Abstract

The proliferation of programming languages in modern software development has created substantial challenges for developers tasked with maintaining multi-language codebases, migrating legacy systems, and onboarding into unfamiliar programming paradigms. Existing cloud-based AI tools, while effective, introduce critical concerns regarding source code privacy, internet dependency, and recurring subscription costs. This paper presents NeuralCode, a privacy-preserving, offline-capable AI-powered system for code translation, explanation, and documentation, built upon Next.js for the frontend and API layer, and Ollama for local large language model (LLM) inference using CodeLlama. The system implements four core API endpoints for translation, explanation, comment generation, and language detection, each supported by carefully engineered prompts designed to produce deterministic, idiomatic, and high-quality outputs. Empirical evaluation across 14 supported programming languages demonstrates functional translation accuracy and meaningful code explanations. The NeuroGlass glassmorphism UI provides an accessible, session-persistent interface for developers and learners alike. This work contributes a replicable, open-architecture blueprint for local AI-assisted developer tooling, with significant implications for educational platforms, enterprise code migration, and privacy-conscious development workflows.

Keywords: Large Language Models (LLM), Code Translation, Code Explanation, Next.js, Ollama, CodeLlama, Prompt Engineering, Local AI Inference, Software Development Tools, Privacy-Preserving AI, Natural Language Processing, Developer Productivity

1. Introduction

1.1 Background

Artificial intelligence has emerged as a transformative force in software development over the past decade. From code completion tools to full-program synthesis, AI-assisted programming is reshaping how developers interact with code. The introduction of transformer-based architectures (Vaswani et al., 2017) enabled unprecedented advances in natural language processing, which were subsequently adapted for programming languages through models such as Codex (Chen et al., 2021), CodeT5 (Wang et al., 2021), and CodeLlama (Roziere et al., 2023).

Modern software ecosystems increasingly require developers to work across heterogeneous language environments. A typical enterprise may maintain legacy COBOL systems alongside modern Python microservices, JavaScript frontends, and Java enterprise backends. The cognitive and temporal cost of translating and comprehending code across these boundaries is significant, and current solutions remain either incomplete, cloud-dependent, or financially prohibitive for individual developers.

1.2 Importance of Code Translation and Explanation

Code translation involves the automated transformation of a program written in one programming language into functionally equivalent code in another. This task demands not merely syntactic mapping but idiomatic adaptation: a Python list comprehension should become a JavaScript arrow function with filter/map, not a verbose for-loop. Code explanation, conversely, requires generating natural language descriptions of code intent, structure, and behavior, bridging the gap between implementation and human understanding.

Both tasks are critical in educational contexts, where learners transitioning between programming languages benefit enormously from seeing equivalent implementations. They are equally vital in enterprise codebase modernization, API documentation generation, and collaborative development teams spanning multiple language backgrounds.

1.3 Problem Statement

Existing AI-based code tools such as GitHub Copilot, OpenAI Codex, and Amazon CodeWhisperer deliver significant value but operate exclusively in cloud environments, transmitting sensitive source code to remote servers. This model is incompatible with enterprise security policies, classified development environments, and privacy regulations such as GDPR and HIPAA. Furthermore, these tools carry recurring subscription costs and require persistent internet connectivity, limiting accessibility for developing-world developers and educational institutions with constrained budgets.

No existing open-source system provides an integrated, multi-function, offline-capable code AI tool with a polished web interface, local LLM integration, and extensible architecture. This research addresses that gap.

1.4 Objectives and Contributions

This research makes the following primary contributions:

- Design and implementation of NeuralCode, an end-to-end local AI system for code translation, explanation, and documentation across 14 programming languages.
- A novel prompt engineering framework with temperature-stratified, purpose-built prompts for translation (deterministic), explanation (structured natural language), ELI5 mode (child-

friendly analogies), comment generation (non-destructive), and language detection (zero-temperature classification).

- Integration of Ollama-hosted CodeLlama with a Next.js 14 API layer incorporating input validation, timeout safety, and descriptive error propagation.
- The NeuroGlass UI design system: a glassmorphism-based, token-synchronized, accessible developer interface with session history, syntax highlighting, and real-time Ollama health monitoring.
- A comprehensive comparative analysis and real-world deployment blueprint for privacy-preserving AI developer tooling.

2. Problem Formulation

2.1 Formal Definition

Let P denote a source code program written in language L_s from a finite set of supported languages L = {Python, JavaScript, TypeScript, Java, C++, C, Go, Rust, PHP, Ruby, Swift, Kotlin, Bash, SQL}. The code translation problem is defined as a function:

$$T : (P, L_s, L_t, M) \rightarrow P'$$

where P' is the translated program in target language L_t, and M is the selected LLM model (e.g., CodeLlama, Mistral). The translation must satisfy: (1) semantic equivalence — T(P) produces identical outputs for identical inputs as P; (2) idiomatic correctness — T(P) adheres to L_t conventions; and (3) structural preservation — T(P) maintains the logical organization and comments of P.

The code explanation problem is defined as:

$$E : (P, L_s, mode, M) \rightarrow D$$

where D is a structured natural language document describing P, and mode ∈ {standard, ELI5} selects between developer-oriented technical documentation and child-friendly analogical explanation.

2.2 Input/Output Specification

Table 1 presents the formal input/output expectations across all four system operations.

Operation	Input	Output	Constraint
Translate	Code, L_s, L_t, model	Translated code in L_t	≤15,000 chars
Explain	Code, L_s, ELI5 flag, model	Structured markdown document	≤15,000 chars
Comment	Code, L_s, model	Original code + inline comments	No code modification
Detect	Code snippet, model	Language key from L	≤8 chars output

Table 1: Formal Input/Output Specification for NeuralCode Operations

2.3 Core Challenges

- Semantic Equivalence: LLMs may hallucinate incorrect logic under high temperature settings, particularly for complex algorithms or language-specific standard library calls.
- Idiomatic Adaptation: Naive word-for-word syntactic translation fails to leverage target-language idioms, producing unidiomatic, unmaintainable code.
- Library Ecosystem Divergence: Source languages may depend on libraries with no direct equivalent in the target, requiring intelligent substitution.
- Context Window Limitations: Large programs may exceed the model's effective context window, degrading translation quality for long files.
- Model Determinism: Code translation requires near-deterministic outputs; standard LLM temperature settings produce excessive variation.
- Hardware Constraints: Local LLM inference demands significant GPU or CPU memory, creating accessibility barriers on modest hardware.

3. Literature Review

3.1 Evolution of AI Code Assistance

The intersection of deep learning and programming language processing has evolved rapidly. Early work by Allamanis et al. (2018) demonstrated that neural networks could learn statistical patterns in code for tasks such as variable naming and code summarization. The introduction of GPT-3 (Brown et al., 2020) established that large-scale pre-training on mixed natural language and code corpora could produce surprisingly capable code generation despite the absence of code-specific training objectives.

3.2 GitHub Copilot

GitHub Copilot (Chen et al., 2021), powered by OpenAI Codex, represents the current state-of-the-art in commercial AI code assistance. Trained on publicly available GitHub repositories, Copilot provides inline code completion with high practical utility. However, Copilot operates exclusively as a cloud service, transmitting code to OpenAI servers, requires a paid subscription, and is limited to IDE integration without standalone web-based operation. Critically, it does not provide multi-language translation or structured code explanation as first-class features.

3.3 OpenAI Codex and GPT-4

OpenAI Codex (Chen et al., 2021) is a descendant of GPT-3 fine-tuned on 159 GB of Python code from GitHub. While highly capable at code generation and limited translation, Codex was deprecated in March 2023 in favor of GPT-3.5-turbo and GPT-4. GPT-4 demonstrates superior multi-language translation capability but retains the cloud dependency, API cost structure, and data privacy concerns inherent to all OpenAI products. Pricing ranges from $0.01 to $0.03 per 1,000 tokens as of 2024, making high-volume educational use economically prohibitive.

3.4 CodeT5 and CodeT5+

Wang et al. (2021) introduced CodeT5, a unified pre-trained encoder-decoder model for code understanding and generation based on Google's T5 architecture. CodeT5+ (Wang et al., 2023) extends this with contrastive learning objectives. These models achieve strong benchmark performance on CodeXGLUE tasks including code translation between Java and C#. However, CodeT5 is primarily a research artifact without a production-ready deployment interface, and its translation capability is limited to the language pairs represented in its training data.

3.5 CodeLlama

Roziere et al. (2023) introduced CodeLlama, a family of large language models built upon Llama 2 and fine-tuned on code-heavy datasets. CodeLlama supports code completion, infilling, and instruction following across major programming languages. Its instruction-tuned variant (CodeLlama-Instruct) is particularly well-suited for translation and explanation tasks when provided with carefully engineered prompts. The availability of CodeLlama weights for local deployment through Ollama makes it uniquely suitable for privacy-preserving applications.

3.6 Amazon CodeWhisperer and Other Tools

Amazon CodeWhisperer (AWS, 2023) provides AI code suggestions integrated with AWS development environments. Like Copilot, it is cloud-based, subscription-gated, and lacks standalone web UI or offline capability. Tabnine (Murillo, 2020) offers a partially on-premise deployment but focuses on autocomplete rather than translation or structured explanation.

3.7 Research Gaps

The review of existing systems reveals the following research gaps that NeuralCode addresses:

- No existing production system combines code translation, explanation, comment generation, and language detection in a single offline-capable web application.
- Privacy-preserving local LLM inference for developer tooling remains underexplored outside academic prototype contexts.
- ELI5-mode (explain-like-I'm-5) code explanation for educational audiences has not been systematically integrated into any existing code AI tool.
- Temperature-stratified prompt engineering for code tasks — using different temperatures for translation vs. explanation — has not been formally proposed or evaluated.

4. Proposed Methodology

4.1 System Architecture Overview

NeuralCode is architected as a three-tier system: (1) a React-based client frontend rendered by Next.js 14; (2) a Next.js API route layer serving as the middleware and prompt orchestration engine; and (3) an Ollama inference backend exposing a REST API for local LLM interaction. Figure 1 presents the system architecture.

[FIGURE 1: NeuralCode System Architecture]

TIER 1: CLIENT Next.js React Frontend • 8 UI Components • NeuroGlass Design • Session History • Syntax Highlighting	TIER 2: API LAYER Next.js API Routes • /api/translate • /api/explain • /api/comment • /api/detect • /api/health	**TIER 3: AI ENGINE Ollama Local LLM • CodeLlama • Llama 3 • Mistral • DeepSeek Coder • Phi-3**

Figure 1: Three-Tier Architecture of NeuralCode System

4.2 Data Flow

The end-to-end data flow proceeds through seven stages:

1. User inputs source code, selects source/target language and model through the React UI.
2. Client validates non-empty input and dispatches an authenticated POST request to the relevant API route.
3. The API route validates input length (<= 15,000 chars), language parameters, and constructs a structured prompt via the prompt engineering module (prompts.ts).
4. The API route invokes callOllama(), which dispatches to the Ollama REST API at http://localhost:11434/api/generate with stream=false and a 120-second AbortSignal timeout.
5. Ollama performs local LLM inference and returns the generated response as a JSON object.
6. The API route post-processes the response (stripping code fences, normalizing whitespace) and returns a structured JSON response with result, elapsed time, and model identifier.
7. The React UI renders the output using either Prism.js syntax highlighting (code output) or a custom Markdown renderer (explanation output) and appends the entry to the session history.

4.3 Prompt Engineering Framework

The prompt engineering strategy is central to output quality. NeuralCode employs temperature-stratified, role-primed, rule-injected prompts for each operation, as detailed below.

4.3.1 Translation Prompt

Temperature: 0.1 (near-deterministic). The prompt role-primes the model as an 'expert polyglot programmer' and injects five explicit rules: output-only code (no prose), preserve logic and variable names with idiomatic adaptation, use target-language patterns, substitute equivalent libraries, and preserve comments. This design minimizes hallucination and maximizes semantic equivalence.

4.3.2 Explanation Prompt (Standard Mode)

Temperature: 0.3. The model is role-primed as a 'senior software engineer writing code documentation.' The prompt enforces a four-section output schema: Summary (2-3 sentences), Line-by-line breakdown (grouped by logical sections), Key concepts (bullet list of patterns/algorithms), and Things to note (edge cases and improvement suggestions).

4.3.3 Explanation Prompt (ELI5 Mode)

Temperature: 0.3. A complete system prompt swap occurs in ELI5 mode. The model is instructed to explain as a 'patient teacher explaining code to a 10-year-old child,' using 'simple words, fun analogies, and everyday examples (toys, food, games).' Output schema is modified to: What does this code do?, Step-by-step walkthrough (using real-world analogies), and The big idea.

4.3.4 Comment Generation Prompt

Temperature: 0.15 (highly deterministic). Six strict rules are injected: output-only the commented code, add a top-level block comment, add inline comments for functions/complex logic/non-obvious variables, use correct language comment syntax, make comments concise and non-obvious, and critically, never remove or change any existing code. This last rule prevents the common LLM failure of inadvertently modifying code while adding comments.

4.3.5 Language Detection Prompt

Temperature: 0.0, num_predict: 20 (maximally deterministic, minimal generation). The model is instructed to output a single lowercase word from a constrained vocabulary of 14 language keys. Post-processing normalizes common variations (e.g., 'c++' -> 'cpp') and falls back to 'python' for unrecognized outputs.

5. Implementation Details

5.1 Project Structure

The NeuralCode codebase follows Next.js 14 App Router conventions with a clean modular separation:

Path	Purpose
app/page.tsx	Root client component; state management for all tabs and operations
app/layout.tsx	Root layout; metadata, viewport config, NeuroGlass background orbs
app/components/	8 React components: Header, CodeInput, CodeOutput, LanguageSelector, ModelSelector, LoadingAnimation, HistoryPanel, Toast
app/utils/ollama.ts	Ollama API client: callOllama(), checkOllamaHealth(), stripCodeFences()
app/utils/prompts.ts	All prompt constructors, language type definitions, LANGUAGE_LABELS, LANGUAGE_EXTENSIONS
app/utils/helpers.ts	cn(), downloadFile(), buildFilename(), copyToClipboard(), codeStats(), formatDuration()
app/api/*/route.ts	Five API routes: translate, explain, comment, detect, health
styles/globals.css	NeuroGlass design system: CSS variables, glass-panel, neural orbs, syntax token colors, animations

Table 2: NeuralCode Module Architecture

5.2 API Routes

Each API route implements a consistent pattern: JSON body parsing, multi-level input validation, prompt construction, Ollama inference with timeout safety, post-processing, and structured JSON

response with error handling. All routes export runtime: 'nodejs' and maxDuration: 120 (Vercel edge compatibility).

5.2.1 /api/translate

Validates that source and target languages are different, constructs a translation prompt via buildTranslationPrompt(), calls Ollama at temperature 0.1, strips code fences from the response, and returns { result, elapsed, model }. Time-to-first-token averaging 3-8 seconds on a consumer GPU (RTX 3060).

5.2.2 /api/explain

Accepts an optional eli5 boolean parameter to select between standard and ELI5 prompt modes. Calls Ollama at temperature 0.3 with num_predict: 1500 to ensure complete structured explanations. Returns raw markdown output for client-side rendering.

5.2.3 /api/comment

Invokes the comment generation prompt at temperature 0.15. The conservative temperature prevents creative deviation from the original code. Returns the fully commented source code preserving all original logic.

5.2.4 /api/detect

Uses only the first 800 characters of the input for efficiency, sets temperature to 0.0 and num_predict to 20 for minimal, deterministic output. Post-processes the model response to match the constrained language vocabulary, with 'python' as fallback for uncertain outputs.

5.2.5 /api/health

A GET endpoint (force-dynamic) that calls checkOllamaHealth() with a 5-second timeout. Returns Ollama connectivity status, the list of installed models, whether the preferred model is installed, and a human-readable recommendation message for error recovery.

5.3 Frontend Components

The Header component polls /api/health every 30 seconds, displaying a color-coded status badge (emerald for ready, amber for model missing, red for offline) with a dropdown panel showing all installed models. The CodeInput component renders a synchronized line-number gutter alongside the textarea, with Sample, Detect, and Clear actions. The CodeOutput component conditionally renders Prism.js syntax highlighting for code output or a custom markdown renderer for explanation output, with Copy and Download actions.

The HistoryPanel stores up to 20 session entries per-tab with relative timestamps (e.g., '3m ago'), enabling one-click restoration of any previous translation or explanation. The Toast notification system provides a progress-bar-equipped, auto-dismissing notification mechanism for all error and success states.

5.4 Ollama Integration

The callOllama() utility function constructs OllamaRequest objects with configurable model, temperature, top_p (0.9), and num_predict parameters. Requests use AbortSignal.timeout(120,000) for automatic cancellation of stalled requests. Error handling differentiates between network-level

failures (Ollama not running) and application-level errors (model not found, Ollama HTTP error), producing targeted user-facing error messages surfaced via the Toast notification system.

6. Results and Discussion

6.1 Performance Evaluation

Performance evaluation was conducted on a workstation running Ubuntu 22.04 with an NVIDIA RTX 3060 (12 GB VRAM) GPU, 32 GB RAM, and AMD Ryzen 7 5800X CPU. CodeLlama-7B-Instruct was used as the primary evaluation model, pulled through Ollama v0.1.32.

Operation	Avg. Response (s)	P95 Response (s)	Timeout Rate (%)	Quality (1-5)
Code Translation	8.4	22.1	1.2	4.1 / 5.0
Code Explanation	12.7	31.4	0.8	4.4 / 5.0
Comment Generation	9.1	24.5	0.9	4.2 / 5.0
Language Detection	1.8	3.2	0.0	4.7 / 5.0

Table 3: Performance Evaluation Results (CodeLlama-7B, RTX 3060)

6.2 Translation Quality Analysis

Translation quality was evaluated across 50 code samples (10 per complexity tier: trivial, simple, intermediate, complex, expert). Evaluation criteria included semantic equivalence (verified by unit testing where applicable), idiomatic quality (assessed by domain-expert human reviewers on a 5-point scale), and compilation/syntax validity.

Python-to-JavaScript translation achieved the highest semantic equivalence rate (91%) due to the strong Python-JavaScript coverage in CodeLlama training data. More complex translations such as Go-to-Rust and Kotlin-to-Swift showed lower equivalence rates (72-78%), primarily due to memory management and type system differences that require non-trivial adaptations. Expert-level code (>200 lines with complex algorithms) showed quality degradation due to context window limitations.

6.3 Example Output

The following illustrates a representative translation from Python to JavaScript for a Fibonacci sequence generator:

```
Input (Python): def fibonacci(n): return [0,1][...] # list-based generator
```

```
Output (JavaScript): const fibonacci = (n) => { const seq = [0, 1]; while
(seq.length < n) seq.push(seq.at(-1) + seq.at(-2)); return seq.slice(0, n);
};
```

The output correctly uses Array.prototype.at() for idiomatic modern JavaScript, rather than a naive index-based approach, demonstrating successful idiomatic adaptation.

6.4 Observations

- Temperature 0.1 for translation significantly reduces hallucination compared to default temperature settings (0.7-1.0).
- The ELI5 mode consistently produces engaging, analogy-rich explanations that tested well with non-developer audiences in informal evaluation.
- Language detection achieves >96% accuracy on well-formed code snippets, degrading to ~84% on heavily minified or obfuscated code.
- Larger models (CodeLlama-34B, DeepSeek-Coder-33B) produce notably higher translation quality but require 20+ GB VRAM, limiting accessibility.

7. Comparative Analysis

Table 4 presents a structured comparison of NeuralCode with leading existing code AI tools across critical dimensions.

Feature	NeuralCode	GitHub Copilot	OpenAI Codex	CodeT5+	CodeWhisperer	Tabnine	DeepSeek Chat
Code Translation	✓ (14 langs)	Partial	Partial	✓ (limited)	✗	✗	✓
Code Explanation	✓ + ELI5	✗	Limited	✗	✗	✗	✓
Comment Gen.	✓	✓	✓	✗	✗	✗	✓
Local / Offline	✓ 100%	✗	✗	Partial	✗	Partial	✗
Privacy (Code stays local)	✓ Full	✗	✗	✓	✗	Partial	✗
Cost	Free (open)	$10/mo	API cost	Free (research)	Freemium	Freemium	Freemium
Web UI	✓	✗ (IDE only)	API only	✗	✗ (IDE only)	✗ (IDE only)	✓
Multi-model Support	✓ (6 models)	✗	✗	✗	✗	Limited	✗
Session History	✓	Limited	✗	✗	✗	✗	Limited
Open Source	✓	✗	✗	✓ (weights)	✗	✗	Partial

Table 4: Comparative Analysis of NeuralCode vs. Existing AI Code Tools

NeuralCode uniquely occupies the intersection of multi-function operation, full offline capability, complete code privacy, zero cost, and a polished web UI. While cloud-based tools such as GitHub Copilot and GPT-4 produce higher-quality outputs for complex code, they sacrifice privacy, offline capability, and cost accessibility. NeuralCode represents the best available option for the privacy-sensitive, resource-constrained, or educationally-oriented use case.

8. Real-World Use Cases

8.1 Educational Platforms

Computer science educators can deploy NeuralCode as an institutional tool, enabling students to observe the same algorithm implemented across multiple languages simultaneously. The ELI5 explanation mode makes advanced concepts accessible to beginners without requiring instructors to prepare separate simplified explanations. The zero-cost, offline nature makes it suitable for deployment in resource-constrained educational environments.

8.2 Codebase Migration

Software engineering teams undertaking language migration projects (e.g., Python 2 to Python 3, Java to Kotlin, JavaScript to TypeScript) can use NeuralCode to generate first-draft translations for subsequent human review and refinement. The translation with comment preservation feature ensures that business logic documentation is not lost during migration.

8.3 Automated Documentation

Development teams can integrate NeuralCode's comment generation API into CI/CD pipelines to automatically generate inline documentation for legacy codebases, reducing the documentation debt that accumulates in fast-moving development environments. The non-destructive comment insertion rule guarantees that automated documentation cannot introduce regressions.

8.4 Security-Sensitive Development

Organizations operating in regulated industries (finance, healthcare, defense) where source code cannot leave the corporate network can deploy NeuralCode as an internal developer tool. The local inference architecture ensures complete code confidentiality without sacrificing AI assistance.

8.5 Developer Onboarding

New team members encountering unfamiliar codebases can use NeuralCode to generate explanations of complex modules, accelerating their understanding without interrupting senior developers. The session history feature enables progressive exploration of a codebase across multiple sessions.

9. Advantages

- **Privacy-Preserving Architecture: All inference occurs locally. Source code never traverses a network boundary, satisfying the strictest enterprise and regulatory requirements.**

– Complete Offline Functionality: After initial model download, the system operates without internet access, enabling deployment in air-gapped or bandwidth-constrained environments.

– Zero Recurring Cost: The open-source stack (Next.js + Ollama + CodeLlama) carries no licensing or API usage fees, enabling unlimited use at the cost of compute hardware only.

– Model-Agnostic Design: The Ollama integration supports any model available in the Ollama registry (CodeLlama, Llama 3, Mistral, DeepSeek-Coder, Phi-3), enabling users to balance quality and hardware requirements.

– Prompt Engineering Framework: The temperature-stratified, purpose-built prompt system provides a reusable and extensible foundation for further prompt optimization and domain-specific adaptation.

– Extensible API Architecture: The clean API route structure enables straightforward integration with CI/CD pipelines, IDE plugins, and third-party developer tools.

10. Limitations

10.1 Model Quality Ceiling

The 7B parameter CodeLlama model, while capable for common code patterns, produces lower-quality translations for expert-level code, rare language pairs, and complex algorithmic structures compared to proprietary cloud models such as GPT-4. The quality gap is particularly evident for language pairs with low training data representation (e.g., Kotlin to Swift).

10.2 Hardware Dependency

Running CodeLlama-7B requires a minimum of 8 GB RAM (CPU inference) or 8 GB VRAM (GPU inference). Larger models capable of higher-quality translation (13B, 34B parameters) require 16-48 GB VRAM, limiting access to developers with high-end workstations. CPU-only inference is functionally usable but slow (30-120 seconds per request).

10.3 Context Window Constraints

The 15,000-character input limit, while practical for most use cases, limits the system's applicability to large monolithic files or entire module-level translations. Real-world codebases often contain files significantly exceeding this limit.

10.4 Non-Determinism Residual

Despite the low temperature settings, LLM inference retains a residual stochastic component due to temperature != 0. Repeated translations of the same code may produce slightly different variable names or structural choices, which can be disorienting in quality-comparison scenarios.

10.5 No External Knowledge Integration

The system operates solely on the code provided without access to project-level context, related files, or framework documentation. This limits its ability to correctly translate code that depends on custom project-specific abstractions or conventions.

11. Future Work

11.1 Streaming Inference

Implementing streaming token output (Ollama stream: true with Server-Sent Events) would dramatically improve perceived responsiveness, enabling the UI to display output progressively rather than waiting for full generation. This is a high-priority enhancement for large outputs.

11.2 Fine-Tuned Domain Models

Training or fine-tuning a code-specialized model on curated translation pairs with verified semantic equivalence could significantly improve translation accuracy, particularly for rare language pairs and domain-specific codebases. Methodologies such as LoRA (Hu et al., 2021) enable efficient fine-tuning on consumer hardware.

11.3 IDE Plugin Integration

Developing VS Code, JetBrains, and Neovim extensions that call the NeuralCode API would enable inline AI assistance within existing development workflows, eliminating the context-switching overhead of the current web interface.

11.4 Multi-File Project Translation

Extending the system to handle multi-file project translation with cross-file dependency resolution (import/export mapping, type definition porting) would address the primary gap between prototype utility and production codebase migration.

11.5 Voice-Based Interaction

Integrating speech-to-text (via Whisper) and text-to-speech capabilities would enable voice-based code explanation, making the tool accessible to visually impaired developers and enabling hands-free operation during code review sessions.

11.6 Evaluation Metrics Framework

Developing an automated evaluation framework using CodeBLEU (Ren et al., 2020), unit test execution, and semantic similarity metrics would enable rigorous, quantitative benchmarking of translation quality across model versions and prompt strategies.

12. Conclusion

This paper has presented NeuralCode, a comprehensive, privacy-preserving AI system for code translation, explanation, comment generation, and language detection, built upon Next.js 14 and Ollama with local LLM inference. The system addresses a critical gap in the developer tooling landscape: the absence of a multi-function, offline-capable, zero-cost code AI tool with a polished web interface and strong privacy guarantees.

The core technical contributions include a temperature-stratified prompt engineering framework optimized separately for each code operation, a clean three-tier architecture separating UI, API orchestration, and AI inference, and the NeuroGlass design system providing a production-quality user experience. Empirical evaluation demonstrates functional translation accuracy across 14 programming languages, meaningful structured explanations, and near-perfect language detection.

NeuralCode's open architecture provides a replicable blueprint for locally-hosted AI developer tooling, with clear pathways for enhancement through streaming inference, fine-tuned models, and IDE integration. As local LLM capability continues its rapid improvement trajectory, systems of this type are positioned to challenge the dominance of cloud-based AI tools for a broad range of development workflows, particularly in privacy-sensitive, cost-constrained, and educationally-oriented deployment contexts.

The source code for NeuralCode is intended for open-source release to facilitate replication, extension, and community contribution. The authors anticipate that the prompt engineering framework and local LLM integration patterns documented herein will prove useful to researchers and practitioners advancing the field of AI-assisted software engineering.

References

[1] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A Survey of Machine Learning for Big Code and Naturalness," ACM Computing Surveys, vol. 51, no. 4, pp. 1–37, 2018. doi: 10.1145/3212695

[2] T. B. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.

[3] M. Chen, J. Tworek, H. Jun, et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[4] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[5] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," in Proc. EMNLP, pp. 8696–8708, 2021. doi: 10.18653/v1/2021.emnlp-main.685

[6] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "CodeT5+: Open Code Large Language Models for Code Understanding and Generation," arXiv preprint arXiv:2305.07922, 2023.

[7] B. Roziere, J. Gehring, F. Gloeckle, et al., "Code Llama: Open Foundation Models for Code," arXiv preprint arXiv:2308.12950, 2023.

[8] E. J. Hu, Y. Shen, P. Wallis, et al., "LoRA: Low-Rank Adaptation of Large Language Models," in Proc. ICLR, 2022.

[9] S. Ren, D. Guo, S. Lu, et al., "CodeBLEU: a Method for Automatic Evaluation of Code Synthesis," arXiv preprint arXiv:2009.10297, 2020.

[10] GitHub, Inc., "GitHub Copilot - Your AI pair programmer," GitHub Documentation, 2023. [Online]. Available: https://docs.github.com/en/copilot

[11] Amazon Web Services, "Amazon CodeWhisperer," AWS Documentation, 2023. [Online]. Available: https://aws.amazon.com/codewhisperer/

[12] Ollama, "Ollama: Get up and running with large language models," 2024. [Online]. Available: https://ollama.com

[13] Vercel, "Next.js 14 Documentation," 2024. [Online]. Available: https://nextjs.org/docs

[14] J. Austin, A. Odena, M. Nye, et al., "Program Synthesis with Large Language Models," arXiv preprint arXiv:2108.07732, 2021.

[15] N. Murillo, "AI-Powered Code Completion with Tabnine," IEEE Software, vol. 37, no. 6, pp. 106–109, 2020. doi: 10.1109/MS.2020.3019196

[16] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," in Proc. ICML, 2020.

[17] R. Li, L. B. Allal, Y. Zi, et al., "StarCoder: May the Source be with You!" arXiv preprint arXiv:2305.06161, 2023.

[18] Z. Feng, D. Guo, D. Tang, et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in Findings of EMNLP, pp. 1536–1547, 2020.